

UNIBUD — Student Dashboard Product

scripts/unibud-verify.ts

```
import { createSpark } from "../src/lib/spark";
import { createBud } from "../src/lib/bud";
import { assembleDashboard, setGoals } from "../src/lib/unibud";

async function main() {
  const spark = createSpark();
  await spark.initialize();
  const bud = createBud({ spark, personality: { name: "Bud", tone: "casual" } });

  const session = { sessionId: "unibud_sl", userId: "student_1", product: "unibud"

  console.log("--- Dashboard with no stated goals yet ---");
  const first = await assembleDashboard(spark, bud, session);
  console.log(JSON.stringify(first, null, 2));
  if (first.suggestedToolId !== null) {
    throw new Error("Expected no suggestion before any goals are set.");
  }

  console.log("\n--- Student sets a goal, asks a real question ---");
  setGoals(spark, session.userId, ["grades", "review"]);
  const second = await assembleDashboard(
    spark,
    bud,
    session,
    "I have a chemistry exam next week, what should I focus on?"
  );
  console.log(JSON.stringify(second, null, 2));

  if (!second.suggestedToolId) {
    throw new Error("Expected a suggested tool once goal tags are set.");
  }
  if (!["study_plans", "assignments"].includes(second.suggestedToolId)) {
    throw new Error(
      `Expected study_plans or assignments to rank highest for ["grades","review"]`
    );
  }

  console.log("\n--- Transcript is readable straight from Bud/Spark, UNIBUD store
```

```

    console.log(bud.transcript(session.sessionId));

    await spark.shutdown();
    console.log("\nUNIBUD dashboard verification completed successfully.");
}

main().catch((err) => {
    console.error("UNIBUD verification failed:", err);
    throw err;
});

```

src/lib/unibud/index.ts

```

export { assembleDashboard } from "./dashboard";
export { setGoals, getGoals } from "./goals";
export { TOOL_DEFINITIONS } from "./tools";
export type { ToolId, ToolDefinition, DashboardViewModel } from "./types";

```

src/lib/unibud/types.ts

```

/**
 * These types belong to UNIBUD only. Spark and Bud know nothing about
 * "tools," "goals," or a "dashboard" – those are UNIBUD's own concepts,
 * built on top of the frozen Spark/Bud SDKs.
 */

export type ToolId =
    | "assignments"
    | "projects"
    | "smart_notes"
    | "study_plans"
    | "campus"
    | "connect";

export interface ToolDefinition {
    id: ToolId;
    name: string;
    description: string;
    tags: string[];
}

```

```

export interface DashboardViewModel {
  greeting: string;
  tools: ToolDefinition[];
  suggestedToolId: ToolId | null;
  suggestedReason: string;
  trace: {
    memoryHits: number;
    knowledgeHits: number;
    reasoningConfidence: number;
    provider: string;
  };
};
}

```

src/lib/unibud/tools.ts

```

import type { ToolDefinition } from "../types";

/**
 * Static configuration, not intelligence. Tags here are matched against
 * a student's stored goal tags (via Spark's Personalization service) to
 * drive the "suggested next" pick on the dashboard.
 */
export const TOOL_DEFINITIONS: ToolDefinition[] = [
  {
    id: "assignments",
    name: "Assignments",
    description: "Track what's due and what's done.",
    tags: ["deadlines", "coursework", "grades"],
  },
  {
    id: "projects",
    name: "Projects",
    description: "Longer-running work with milestones.",
    tags: ["coursework", "collaboration", "deadlines"],
  },
  {
    id: "smart_notes",
    name: "Smart Notes",
    description: "Notes that connect back to your courses.",
    tags: ["coursework", "study", "review"],
  },
  {
    id: "study_plans",
    name: "Study Plans",

```

```

        description: "A sequenced plan for an exam or a subject.",
        tags: ["study", "review", "grades"],
    },
    {
        id: "campus",
        name: "Campus",
        description: "Events, deadlines, and services at your school.",
        tags: ["campus_life", "community"],
    },
    {
        id: "connect",
        name: "Connect",
        description: "Study groups, mentors, and classmates.",
        tags: ["community", "collaboration"],
    },
];

```

src/lib/unibud/goals.ts

```

import type { Spark } from "../spark";

const GOALS_KEY = "academic_goal_tags";

/**
 * UNIBUD-specific convenience over Spark.personalization. Spark itself
 * has no concept of "academic goals" – it just stores arbitrary tagged
 * preferences per user. This is where UNIBUD gives that generic
 * mechanism product meaning.
 */
export function setGoals(spark: Spark, userId: string, tags: string[]): void {
    spark.personalization.setPreference(userId, GOALS_KEY, tags);
}

export function getGoals(spark: Spark, userId: string): string[] {
    const pref = spark.personalization.getPreference(userId, GOALS_KEY);
    return Array.isArray(pref?.value) ? (pref!.value as string[]) : [];
}

```

src/lib/unibud/dashboard.ts

```

import type { Spark } from "../spark";

```

```

import type { Bud } from "../bud";
import type { BudSession } from "../bud";
import type { DashboardViewModel } from "./types";
import { TOOL_DEFINITIONS } from "./tools";
import { getGoals } from "./goals";

/**
 * The landing-page flow described in the product brief:
 *
 * 1. Bud greets the student and reasons over their message (via Bud's
 *    own respond() – UNIBUD does not reimplement conversation).
 * 2. Spark.recommendations ranks UNIBUD's own tool catalog against
 *    the student's stored goal tags – a dashboard-level concern Bud
 *    doesn't expose, so UNIBUD calls Spark directly for it.
 *
 * Bud and Spark's public APIs are used exactly as frozen; nothing here
 * reaches into their internals.
 */
export async function assembleDashboard(
  spark: Spark,
  bud: Bud,
  session: BudSession,
  message?: string
): Promise<DashboardViewModel> {
  const response = await bud.respond(
    message ?? "Greet me and help me figure out what to work on.",
    session
  );

  const goalTags = getGoals(spark, session.userId);

  const ranked = spark.recommendations.recommend({
    userId: session.userId,
    sessionId: session.sessionId,
    candidateItems: TOOL_DEFINITIONS.map((t) => ({ id: t.id, tags: t.tags })),
    basedOnTags: goalTags,
    limit: 1,
  });

  const top = ranked[0];
  const suggestedToolId = top && top.score > 0 ? (top.itemId as DashboardViewMode

  return {
    greeting: response.message,
    tools: TOOL_DEFINITIONS,
    suggestedToolId,
    suggestedReason: top?.reason ?? "Set an academic goal to get a personalized p

```

```

    trace: {
      memoryHits: response.trace.memoryHits,
      knowledgeHits: response.trace.knowledgeHits,
      reasoningConfidence: response.trace.reasoningConfidence,
      provider: response.trace.provider,
    },
  };
}

```

unibud-dashboard.jsx (interactive preview component)

```

import React, { useMemo, useState } from "react";
import {
  BookOpen,
  FolderKanban,
  NotebookPen,
  CalendarClock,
  Building2,
  Users,
  Pin,
  Sparkles,
} from "lucide-react";

/**
 * UNIBUD – student dashboard (visual preview)
 * -----
 * This is a self-contained UI preview for the chat surface. The real
 * data behind it – Bud's greeting, the ranked "focus pick" – is
 * already implemented and verified against the actual Spark/Bud SDKs
 * in src/lib/unibud/dashboard.ts (assembleDashboard()). In production
 * this component would call that function through an API route
 * instead of the local mock logic below, which mirrors the same
 * tag-overlap scoring Spark.recommendations performs so the behavior
 * you see here matches the verified backend.
 */

const PALETTE = {
  ink: "#1B1F3B",
  paper: "#FBF8F2",
  blue: "#4C6FFF",
  coral: "#FF6B5E",
  sage: "#7C9473",
  graphite: "#2B2B2B",
  hair: "#E4DFD3",

```

```
};
```

```
const TOOLS = [  
  {  
    id: "assignments",  
    name: "Assignments",  
    description: "Track what's due and what's done.",  
    tags: ["deadlines", "coursework", "grades"],  
    icon: BookOpen,  
    stat: "3 due this week",  
  },  
  {  
    id: "projects",  
    name: "Projects",  
    description: "Longer-running work with milestones.",  
    tags: ["coursework", "collaboration", "deadlines"],  
    icon: FolderKanban,  
    stat: "1 milestone Friday",  
  },  
  {  
    id: "smart_notes",  
    name: "Smart Notes",  
    description: "Notes that connect back to your courses.",  
    tags: ["coursework", "study", "review"],  
    icon: NotebookPen,  
    stat: "12 notes this term",  
  },  
  {  
    id: "study_plans",  
    name: "Study Plans",  
    description: "A sequenced plan for an exam or a subject.",  
    tags: ["study", "review", "grades"],  
    icon: CalendarClock,  
    stat: "Chem exam in 6 days",  
  },  
  {  
    id: "campus",  
    name: "Campus",  
    description: "Events, deadlines, and services at your school.",  
    tags: ["campus_life", "community"],  
    icon: Building2,  
    stat: "2 events this week",  
  },  
  {  
    id: "connect",  
    name: "Connect",  
    description: "Study groups, mentors, and classmates.",
```

```

    tags: ["community", "collaboration"],
    icon: Users,
    stat: "1 group invite",
  },
];

const GOAL_OPTIONS = [
  { key: "grades", label: "Raise my grades" },
  { key: "review", label: "Get ready for exams" },
  { key: "deadlines", label: "Stop missing deadlines" },
  { key: "community", label: "Find people to work with" },
];

function rankTools(goalTags) {
  if (goalTags.length === 0) return null;
  const scored = TOOLS.map((tool) => {
    const overlap = tool.tags.filter((t) => goalTags.includes(t)).length;
    return { tool, score: overlap / goalTags.length };
  });
  scored.sort((a, b) => b.score - a.score);
  return scored[0].score > 0 ? scored[0] : null;
}

function greetingFor(name, goalTags) {
  const hour = new Date().getHours();
  const timeGreeting = hour < 12 ? "Morning" : hour < 18 ? "Afternoon" : "Evening";
  if (goalTags.length === 0) {
    return `${timeGreeting}, ${name}. Tell me what you're working toward and I'll
  }
  return `${timeGreeting}, ${name}. Based on what you told me, here's where I'd s
}

export default function UnibudDashboard() {
  const [goalTags, setGoalTags] = useState([]);
  const [activeToolId, setActiveToolId] = useState(null);

  const toggleGoal = (key) => {
    setGoalTags((prev) =>
      prev.includes(key) ? prev.filter((k) => k !== key) : [...prev, key]
    );
  };

  const pick = useMemo(() => rankTools(goalTags), [goalTags]);
  const greeting = useMemo(() => greetingFor("Andrew", goalTags), [goalTags]);
  const activeTool = TOOLS.find((t) => t.id === activeToolId) || null;

  return (

```



```

<div
  style={{
    minHeight: "100vh",
    background: PALETTE.paper,
    color: PALETTE.graphite,
    fontFamily: "'IBM Plex Sans', system-ui, sans-serif",
  }}
>
<style>{`
  @import url('https://fonts.googleapis.com/css2?family=Fraunces:wght@400;5
  .unibud-card { transition: transform 160ms ease, box-shadow 160ms ease; }
  .unibud-card:hover { transform: translateY(-3px); box-shadow: 0 10px 24px
  .unibud-pin { animation: unibud-pin-in 420ms cubic-bezier(.2,.9,.3,1.3);
  @keyframes unibud-pin-in {
    from { opacity: 0; transform: rotate(-6deg) translateY(-10px) scale(0.9
    to   { opacity: 1; transform: rotate(-2deg) translateY(0) scale(1); }
  }
  .unibud-chip { transition: background 140ms ease, color 140ms ease, borde
  @media (prefers-reduced-motion: reduce) {
    .unibud-card, .unibud-pin, .unibud-chip { transition: none; animation:
  }
  `}</style>

  {/* Header / Bud greeting */}
<header style={{ background: PALETTE.ink, padding: "28px 20px 40px" }}>
  <div style={{ maxWidth: 720, margin: "0 auto" }}>
    <div
      style={{
        display: "flex",
        alignItems: "center",
        gap: 8,
        marginBottom: 14,
      }}
    >
    <div
      style={{
        width: 30,
        height: 30,
        borderRadius: 999,
        background: PALETTE.coral,
        display: "flex",
        alignItems: "center",
        justifyContent: "center",
        flexShrink: 0,
      }}
    >
    <Sparkles size={16} color={PALETTE.ink} />

```

```

    </div>
    <span
      style={{
        fontFamily: "'IBM Plex Mono', monospace",
        fontSize: 12,
        letterSpacing: "0.08em",
        color: "#9AA3D1",
        textTransform: "uppercase",
      }}
    >
      Bud · UNIBUD
    </span>
  </div>
  <h1
    style={{
      fontFamily: "'Fraunces', serif",
      fontWeight: 500,
      fontSize: "clamp(22px, 5vw, 30px)",
      lineHeight: 1.3,
      color: PALETTE.paper,
      margin: 0,
    }}
  >
    {greeting}
  </h1>
</div>
</header>

{/* Goal chips */}
<div style={{ maxWidth: 720, margin: "0 auto", padding: "20px 20px 0" }}>
  <p
    style={{
      fontFamily: "'IBM Plex Mono', monospace",
      fontSize: 11,
      letterSpacing: "0.06em",
      textTransform: "uppercase",
      color: "#8A8578",
      margin: "0 0 10px",
    }}
  >
    What are you working toward?
  </p>
  <div style={{ display: "flex", flexWrap: "wrap", gap: 8 }}>
    {GOAL_OPTIONS.map((g) => {
      const active = goalTags.includes(g.key);
      return (
        <button

```

```

        key={g.key}
        onClick={() => toggleGoal(g.key)}
        className="unibud-chip"
        style={{
          padding: "7px 14px",
          borderRadius: 999,
          fontSize: 13.5,
          fontFamily: "'IBM Plex Sans', sans-serif",
          fontWeight: 500,
          border: `1px solid ${active ? PALETTE.blue : PALETTE.hair}`,
          background: active ? PALETTE.blue : "transparent",
          color: active ? PALETTE.paper : PALETTE.graphite,
          cursor: "pointer",
        }}
      >
        {g.label}
      </button>
    );
  }}}
</div>
</div>

{/* Focus pick – the signature element */}
<div style={{ maxWidth: 720, margin: "0 auto", padding: "26px 20px 0" }}>
  {pick ? (
    <div
      key={pick.tool.id}
      className="unibud-pin"
      style={{
        transform: "rotate(-2deg)",
        background: "#FFF7DC",
        border: `1px solid ${PALETTE.hair}`,
        borderRadius: 4,
        padding: "16px 18px",
        boxShadow: "0 8px 20px rgba(27,31,59,0.10)",
        position: "relative",
        maxWidth: 420,
      }}
    >
      <Pin
        size={16}
        color={PALETTE.coral}
        style={{ position: "absolute", top: -8, left: 18, transform: "rotat
        fill={PALETTE.coral}
      />
    </p>
    style={{

```

```

        fontFamily: "'IBM Plex Mono', monospace",
        fontSize: 11,
        letterSpacing: "0.06em",
        textTransform: "uppercase",
        color: PALETTE.coral,
        margin: "0 0 6px",
    }}
>
    Bud's pick
</p>
<p style={{ margin: "0 0 4px", fontFamily: "'Fraunces', serif", fontS
    Start with {pick.tool.name}
</p>
<p style={{ margin: 0, fontSize: 13.5, color: "#5B5748" }}>
    {pick.tool.stat} – matches what you told me you're working on.
</p>
</div>
) : (
    <p style={{ fontSize: 13.5, color: "#8A8578", fontStyle: "italic" }}>
        Pick a goal above and I'll point you at one tool to start with.
    </p>
    )}
</div>

{/* Tool grid */}
<main style={{ maxWidth: 720, margin: "0 auto", padding: "26px 20px 60px" }
    <div
        style={{
            display: "grid",
            gridTemplateColumns: "repeat(auto-fit, minmax(150px, 1fr))",
            gap: 14,
        }}
    >
        {TOOLS.map((tool) => {
            const Icon = tool.icon;
            const isPick = pick?.tool.id === tool.id;
            const isActive = activeToolId === tool.id;
            return (
                <button
                    key={tool.id}
                    onClick={() => setActiveToolId(isActive ? null : tool.id)}
                    className="unibud-card"
                    style={{
                        textAlign: "left",
                        background: "#fff",
                        border: `1px solid ${isPick ? PALETTE.blue : PALETTE.hair}`,
                        borderRadius: 10,

```

```

        padding: "16px 16px 14px",
        cursor: "pointer",
        fontFamily: "inherit",
    }}
>
<Icon size={20} color={isPick ? PALETTE.blue : PALETTE.graphite}
<p
    style={{
        margin: "10px 0 4px",
        fontFamily: "'Fraunces', serif",
        fontSize: 16.5,
        color: PALETTE.graphite,
    }}
>
    {tool.name}
</p>
<p style={{ margin: "0 0 10px", fontSize: 12.5, color: "#7A7568",
    {tool.description}
</p>
<p
    style={{
        margin: 0,
        fontFamily: "'IBM Plex Mono', monospace",
        fontSize: 11,
        color: PALETTE.sage,
    }}
>
    {tool.stat}
</p>
</button>
);
    )})
</div>

```

```

{activeTool && (
    <div
        style={{
            marginTop: 18,
            borderTop: `1px solid ${PALETTE.hair}`,
            paddingTop: 16,
        }}
    >
        <p
            style={{
                fontFamily: "'IBM Plex Mono', monospace",
                fontSize: 11,
                letterSpacing: "0.06em",

```

```
        textTransform: "uppercase",
        color: "#8A8578",
        margin: "0 0 6px",
    }}
>
    Opening
</p>
<p style={{ margin: 0, fontFamily: "'Fraunces', serif", fontSize: 18
    {activeTool.name} – not built yet in this preview.
</p>
<p style={{ margin: "6px 0 0", fontSize: 13, color: "#7A7568" }}>
    This is where the {activeTool.name.toLowerCase()} workspace opens n
</p>
</div>
    )}
</main>
</div>
);
}
```